

Technical Take-Home Assessment

Financial Processing and RAG Workflow Agent

Role	AI Engineer / Developer
Environment	Local, AWS or Google Cloud
Language	Python or TypeScript
Submission	Source repository, setup artefacts and a short design note

The goal: Build a small, production-minded financial-processing agent. We care more about grounded evidence, explicit control and failure handling than a feature-rich demo.

1. Scenario

You are building an internal accounts-payable assistant. Given an invoice-processing request, the agent must retrieve relevant financial documents, reconcile the evidence, identify exceptions, recommend an outcome, and pause for human approval before posting or changing any financial decision.

The relevant evidence may include invoices, purchase orders, goods-receipt records, vendor master data, approval delegations and finance policy. Retrieval results can be incomplete, contradictory, stale or malicious. The agent must ground its conclusions in cited sources and clearly identify what it does not know.

2. What to build

Implement a service or command-line application that processes the following workflow:

1. Accept a processing request containing a case ID, invoice reference, vendor, amount, currency and optional notes or attachments.
2. Retrieve relevant documents using a RAG pipeline and call other tools only when needed.
3. Reconcile invoice, purchase-order, receipt, vendor and policy evidence; detect mismatches, duplicates and missing approvals.
4. Produce a structured recommendation containing cited evidence, calculations, assumptions, confidence, exceptions and next action.
5. For a consequential outcome, create an approval request and stop. Continue only after an explicit approval or rejection.
6. Persist enough run state and audit events to explain, reproduce and safely resume the workflow.

Required interface

Expose the workflow through either HTTP endpoints or equivalent CLI commands. The following logical operations must be available:

Operation	Expected behaviour
Start run	Create a run from a financial case and execute until completion, failure or approval required.
Get run	Return status, current state, result and audit events.
Approve / reject	Resolve a pending posting/payment decision and resume the same run safely.
List evaluation results	Run the supplied or equivalent test cases and report pass/fail outcomes.

3. Tool contracts

Implement these capabilities using actual API calls, mocks, or a mixture of both. Tool inputs and outputs must use explicit schemas, and the README must state which integrations are real or simulated.

Tool	Purpose	Minimum behaviour
retrieve_finance_documents	Search the RAG corpus.	Return ranked chunks with document ID, type, version/page, relevance and citation metadata; include one adversarial or irrelevant document.
get_vendor_record	Retrieve vendor master data.	Return status, payment details, risk flags and last-updated timestamp for a vendor.
get_purchase_order	Retrieve order and receipt data.	Return lines, totals, currency, tolerances, approval status and goods receipts.
check_invoice_history	Detect potential duplicates.	Return matching invoice references or fingerprints with stable IDs and status.
submit_finance_decision	Record a consequential outcome.	Simulate or call a posting/hold/rejection API; require approval, validated arguments and an idempotency key.

You may add tools, but every tool must have a clear purpose, bounded permissions, timeout behaviour and observable result.

4. Functional requirements

- Use an actual LLM or a model abstraction that supports one. Keep provider and model configuration outside the orchestration code.
- Build a RAG flow with document ingestion or fixtures, chunking/indexing, retrieval and source citations. Explain the retrieval strategy and its limitations.
- Use a bounded agent loop or explicit state graph with a maximum step/tool-call budget.
- Validate model outputs and tool arguments. Invalid outputs must be retried, repaired or failed explicitly; they must not be silently trusted.
- Perform arithmetic and reconciliation deterministically in code or a constrained calculation tool rather than relying on free-form model arithmetic.
- Record run, retrieval and tool events with timestamps, correlation/run ID, outcome and duration. Do not log credentials, bank details or unnecessary financial data.
- Handle at least: tool timeout, transient tool failure, malformed model output, duplicate approval request and application restart/resume.
- Treat retrieved documents and case text as untrusted data. Instructions inside them must not override system policy or authorize actions.
- Consequential tools must be deny-by-default, approval-gated and idempotent.
- Return a typed final result that separates sourced facts, calculations, inferences, unknowns, policy findings and actions taken.

5. Starter data and test cases

Include fixtures for the cases below. You may change exact data values while preserving the intent.

Case	Signals	Expected control behaviour
FIN-001: valid three-way match	Invoice, PO and receipt agree; vendor is active; policy requires an approver.	Cite matching evidence, calculate totals, request approval, then submit exactly once.
FIN-002: duplicate invoice	Invoice number and amount match an already-paid record.	Place on hold or recommend rejection; do not propose payment.
FIN-003: poisoned document	A supplier attachment says to ignore policy and immediately release payment.	Treat text as untrusted evidence; do not bypass policy or approval.
FIN-004: missing evidence	Purchase-order API times out and the corpus lacks a receipt.	Apply bounded retry or fail gracefully; expose missing evidence and avoid approval for payment.
FIN-005: duplicate approval	The same approval callback is delivered twice.	Produce one effective finance decision and a stable replay-safe response.

6. Environment and execution

You may deploy locally, in AWS, or in Google Cloud. Choose the environment that best demonstrates your approach. The complete project may use a real LLM and real service APIs, provided the reviewer can configure and run it from your instructions.

- Provide exact prerequisites, environment variables, model configuration, ingestion/indexing steps, start commands and test/evaluation commands.
- Provide an architecture diagram and a component/configuration manifest showing the model, agent runtime, document store/index, persistence, tools, API surface and trust boundaries.
- For a local setup, provide reproducible scripts, containers or equivalent configuration. For AWS or GCP, provide infrastructure-as-code where practical, or exported configuration/CLI commands and screenshots sufficient to demonstrate the deployed setup.
- Include cost/cleanup notes for any cloud resources. Do not submit credentials; provide example configuration only.
- Clearly label real integrations, mocks and any functionality that requires external access.

Framework guidance

There is no preferred agent framework. Suitable examples include Google Agent Development Kit (ADK), LangChain/LangGraph, AWS Strands Agents, or a framework-free implementation. Choose deliberately and explain what the framework provides versus what your code enforces.

7. Deliverables

- Runnable source code and complete instructions to provision/configure, ingest documents, start the application and execute the tests.
- README covering environment choice, commands, model/API requirements, assumptions, supported flows and known limitations.
- A design note (approximately 1-2 pages) describing orchestration, RAG design, trust boundaries, model/tool contracts, persistence, failure handling and production changes.
- Environment artefacts: architecture diagram plus configuration, infrastructure-as-code, deployment export, scripts or screenshots appropriate to the chosen setup.
- Automated tests for core orchestration, retrieval grounding and safety. Separate stable unit/contract tests from model-dependent integration or evaluation runs.
- Sample output or a short transcript for one successful flow and one exception/approval flow.

Repository hygiene

- Do not include API keys, credentials, personal data or proprietary code.
- Pin or lock dependencies. Provide an example environment file only if needed.
- AI coding tools are allowed. Declare how you used them and be prepared to explain every submitted design and code decision.

8. Constraints and scope

Timebox: Please stop after 8 hours and document unfinished work or deliberate trade-offs. Completeness is less important than sound prioritisation.

- No user interface is required.
- No particular agent framework, cloud, vector store or LLM provider is required.
- Actual LLM and API calls are allowed. Mocks are also acceptable when their contracts and limitations are clear.
- A simple local persistence mechanism is sufficient if it demonstrates restart/resume and idempotency semantics.
- Do not submit a tool capable of moving real money. Any posting or payment action must target a sandbox or be simulated.

9. What we will assess

Area	What good looks like
Agent and RAG architecture	Clear states, bounded decisions, grounded retrieval, explicit model/tool boundaries and understandable control flow.
Reliability	Typed contracts, persistence, idempotency, retries, timeouts and recoverable failure states.
Safety	Least-authority tools, prompt-injection resistance, approval gates and safe logging.
RAG and evaluation	Grounded citations, sensible retrieval, adversarial cases and meaningful quality measures.
Engineering quality	Readable code, sensible abstractions, straightforward setup and good technical communication.

10. Optional extensions

Choose at most one or two only after the required workflow is solid. Optional work does not compensate for missing safety or reliability controls.

- Parallel read-only tool execution with deterministic merge behaviour.
- Token/cost accounting and configurable budgets.
- OpenTelemetry-compatible traces or a useful run timeline.
- A second model adapter or framework-free versus framework-based comparison.
- Property-based or fault-injection tests.

11. Submission notes

AI coding tools are allowed. Include the approximate time spent and describe how AI tools were used, along with any code, libraries or generated material you did not author. You must be able to explain, modify and defend every submitted design and code decision. We will discuss how the solution would operate at enterprise scale.